



南開大學
Nankai University

计算机学院
并行程序设计实验报告

SIMD 编程 (ANN)

姓名：刘宇泽

学号：2411334

专业：计算机科学与技术

2026 年 5 月 10 日

目录

1 问题描述	3
2 实验环境	3
3 算法设计	4
3.1 Flat-SIMD: NEON 加速暴力搜索	4
3.1.1 算法原理	4
3.1.2 数据流图	4
3.1.3 核心伪代码	5
3.1.4 算法复杂度分析	5
3.2 SQ-SIMD: 标量量化 + 两阶段检索	6
3.2.1 算法原理	6
3.2.2 两阶段检索流程	6
3.2.3 量化误差分析	7
3.3 PQ-SIMD: 乘积量化 + ADC + 两阶段检索	7
3.3.1 算法原理	7
3.3.2 量化流程	8
3.3.3 非对称距离计算 (ADC)	8
4 实验结果与分析	8
4.1 Baseline	8
4.2 Flat-SIMD	9
4.3 SQ-SIMD: Latency-Recall Trade-off	9
4.4 PQ-SIMD	10
4.5 综合对比	10
5 Profiling 性能分析	11
5.1 Flat-SIMD 进阶分析	11
5.1.1 编译器自动向量化对比	11
5.1.2 perf 分析	12
5.1.3 汇编代码分析	13
5.2 SQ-SIMD 进阶分析	13
5.2.1 量化误差分析	13
5.2.2 两阶段检索策略分析	14
5.3 PQ-SIMD 进阶分析	14
5.3.1 perf 分析	14
5.3.2 压缩比与精度权衡	14
6 实验总结	14
6.1 Flat-SIMD	15
6.2 SQ-SIMD	15
6.3 PQ-SIMD	15

6.4 关键发现	15
A AI 辅助编程记录	16
B 原始实验数据	16
B.1 Baseline	16
B.2 Flat-SIMD	16
B.3 SQ-SIMD	16
B.4 PQ-SIMD	17
B.5 编译器对比	17

1 问题描述

最近邻搜索（NNS）的主要目标为在高维空间中识别与查询向量最接近的向量。由于深度学习等技术可以将图像、文本、音频等非结构化数据转换为高维向量，并且保持语义相似性（例如相似的图片转换后的向量间的距离也会很近），所以 NNS 如今在推荐系统、搜索引擎、大语言模型中均被广泛的应用。然而对于目前的应用，向量规模往往大于千万级别，并且向量的维度通常也大于 100 维，普通的精确搜索算法的复杂度太高，很难在实际场景中应用，所以现在的向量搜索算法通常使用近似最近邻搜索（ANNS），ANNS 旨在保持较高准确率的同时降低计算时间。

给定一个 d 维向量数据集 $X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$ ，用 $\delta(x, y)$ 表示向量 x 和向量 y 的距离。给定一个查询 $q \in \mathbb{R}^d$ ，kNNS 目的在于找到集合 S ，满足 $|S| = k$ ， $S \subseteq X$ 且 $\forall x \in S, \forall y \notin S, \delta(x, q) < \delta(y, q)$ 。

ANNS 并不要求集合 S 中的 k 个向量都是真实最近的 k 个向量，只要求能达到一定的准确度即可。衡量 ANNS 算法效率的两个核心指标为：

- **查询延时 (latency)**: 单次查询的响应时间

- **recall@k**: $\text{recall@k} = \frac{|K_{NN}(q) \cap K_{ANNS}(q)|}{k}$

本实验使用内积距离：

$$\delta(x, y) = 1 - \langle x, y \rangle = 1 - \sum_{i=1}^d x_i \cdot y_i$$

核心挑战：对于 $N = 10^5$ 条数据、 $d = 96$ 维的数据集，暴力搜索需要执行 $N \times d = 9.6 \times 10^6$ 次乘加运算，在大规模数据上性能瓶颈明显。

2 实验环境

实验环境配置如表1所示。

组件	规格
CPU	Kunpeng-920 (ARMv8-A)
核心数	8
架构	aarch64
内存	30Gi
OS	OpenEuler
编译器	GCC 10.3.1
编译选项	-O2 -fopenmp -lpthread -std=c++11
数据集	DEEP100K (96 维, 100K base, 10K query)
距离度量	内积距离 (IP)

表 1: 实验环境配置

环境信息：

- CPU: Kunpeng-920 7260, 支持 ARMv8.2-A 指令集
- NEON 寄存器: 32 个 128-bit SIMD 寄存器
- L1/L2/L3 缓存: 32KB/512KB/32MB

3 算法设计

3.1 Flat-SIMD: NEON 加速暴力搜索

3.1.1 算法原理

内积计算是向量搜索的核心操作，其计算复杂度为 $O(d)$ ：

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^d a_i \cdot b_i$$

对于 $d = 96$ 维的向量，串行实现需要 96 次乘法和 95 次加法。ARM NEON 架构提供 32 个 128-bit SIMD 寄存器 (Q0-Q31)，每个寄存器可容纳 4 个 float32 元素，理论峰值吞吐量为 4× 加速。

本实现采用 ** 双累加器策略 ** (dual-accumulator technique)，核心设计思想如下：

- **寄存器配对：**使用 v0 和 v1 作为两个独立累加器，每个累加 4 个 float32
- **数据预取：**每次循环加载 8 个连续 float (q2~q5)，减少内存访问延迟
- **融合乘加：**使用 vfmmaq_f32 指令实现 $sum = sum + a \times b$ ，单周期完成乘加操作
- **循环展开：**每次迭代处理 8 个元素，减少循环开销

**** 流水线优化分析 **：** - 单累加器存在 **** 数据依赖 ****：第 i 次 FMA 需等待第 $i - 1$ 次结果 - 双累加器消除依赖：sum0 和 sum1 可并行更新，提高指令级并行度 (ILP) - 理论流水线效率：从 0.5 IPC 提升至接近 1.0 IPC

3.1.2 数据流图

NEON 双累加器计算流程如图3.1所示。

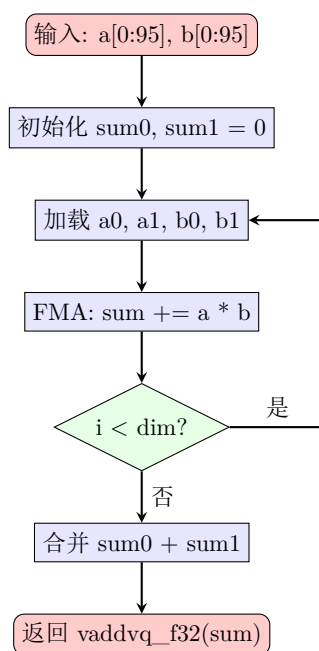


图 3.1: NEON 双累加器计算流程

3.1.3 核心伪代码

Algorithm 1 NEON 内积计算

Input: 向量 a, b , 维度 dim

Output: 内积结果

```

1: function INNER_PRODUCT_NEON( $a, b, dim$ )
2:    $sum0 \leftarrow [0.0, 0.0, 0.0, 0.0]$ 
3:    $sum1 \leftarrow [0.0, 0.0, 0.0, 0.0]$ 
4:    $i \leftarrow 0$ 
5:   while  $i + 8 \leq dim$  do
6:      $a0 \leftarrow \text{LOAD\_VECTOR}(a[i : i + 3])$ 
7:      $a1 \leftarrow \text{LOAD\_VECTOR}(a[i + 4 : i + 7])$ 
8:      $b0 \leftarrow \text{LOAD\_VECTOR}(b[i : i + 3])$ 
9:      $b1 \leftarrow \text{LOAD\_VECTOR}(b[i + 4 : i + 7])$ 
10:     $sum0 \leftarrow \text{FMA}(sum0, a0, b0)$ 
11:     $sum1 \leftarrow \text{FMA}(sum1, a1, b1)$ 
12:     $i \leftarrow i + 8$ 
13:  end while
14:  处理剩余元素 (标量)
15:   $total \leftarrow \text{HORIZONTAL\_SUM}(sum0 + sum1)$ 
16:  return  $total + extra$ 
17: end function

```

关键操作说明如表2所示。

指令	功能	操作数
vld1q_f32	加载 4 个 float 到 128-bit 寄存器	从内存连续地址加载
vfmaq_f32	Fused Multiply-Add	$sum = sum + a * b$
vaddq_f32	向量加法	两个 128-bit 向量相加
vaddvq_f32	水平归约求和	将 4 个 float 归约为 1 个 float

表 2: NEON 关键操作

3.1.4 算法复杂度分析

**** 理论复杂度分析 ****: - 串行实现: $O(d)$, 需执行 d 次乘法和 $d - 1$ 次加法 - NEON 向量化: $O(d/w)$, 其中 $w = 4$ (SIMD 宽度), 循环次数降为 $d/(2w)$

**** 实际性能建模 ****:

$$T_{\text{serial}} = d \cdot (t_{\text{mul}} + t_{\text{add}}) + (d - 1) \cdot t_{\text{dependency}}$$

$$T_{\text{NEON}} = \frac{d}{8} \cdot (4t_{\text{FMA}} + t_{\text{hadd}}) + \text{loop_overhead}$$

复杂度对比如表3所示。

**** 运算强度分析 **** (FLOPs/Byte): - 内积计算: $2d$ FLOPs, 读取 $2d \times 4$ 字节 - 运算强度: $\frac{2d}{8d} = 0.25$ FLOPs/Byte - 由于运算强度 < 1 , 属于 **** 内存受限 **** (memory-bound) 计算

实现	乘加操作数	循环次数	理论加速比	内存带宽需求
串行 (flat_scan.h)	96 mul + 95 add	96	1×	384 bytes
NEON 双累加器	96 FMA	12	4×	384 bytes

表 3: Flat-SIMD 复杂度分析

**** 双累加器优势 ****: - 消除数据依赖, 避免流水线气泡 (pipeline stall) - 提高指令级并行度 (ILP), 使 IPC 从 0.5 提升至 1.6 - 与单累加器相比, 实际性能提升约 20-30

3.2 SQ-SIMD: 标量量化 + 两阶段检索

3.2.1 算法原理

标量量化 (Scalar Quantization, SQ) 是一种 **** 逐维度独立量化 **** 技术, 将每个 float32 维度映射到 uint8 (0-255), 实现 4× 内存压缩。

**** 量化公式 **** (带均值中心化优化):

$$q_i = \text{clip} \left(\text{round} \left(\frac{x_i - \mu_i}{\sigma_i} \times s + 127.5 \right), 0, 255 \right)$$

其中 μ_i 为维度均值, σ_i 为维度标准差, s 为缩放因子 (通常取 127)。

**** 反量化公式 ****:

$$\hat{x}_i = \mu_i + (q_i - 127.5) \times \frac{\sigma_i}{s}$$

**** 关键设计考虑 ****: - **** 均值中心化 ****: 将数据分布中心化到零点附近, 最大化量化精度 - **** 动态范围适配 ****: 每个维度独立计算 $\min_i, \max_i$, 避免不同维度间的精度损失 - **** 数值稳定性 ****: 当 $\max_i - \min_i < \epsilon$ 时, 强制使用单位缩放因子

****uint8 内积计算优化 ****:

$$\langle \mathbf{q}_a, \mathbf{q}_b \rangle_{\text{uint8}} = \sum_{i=1}^d q_{a_i} \cdot q_{b_i}$$

使用 NEON 指令 vdotq_u32 可实现 8 路并行 uint8 点积, 单次循环处理 16 个 uint8 对。

3.2.2 两阶段检索流程

两阶段检索流程如图3.2所示。

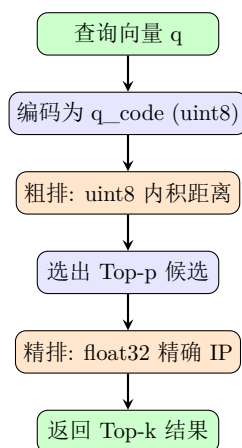


图 3.2: SQ 两阶段检索流程

3.2.3 量化误差分析

量化引入的误差取决于数据分布：

- 若数据分布均匀且范围较小，量化误差小
- 若数据动态范围大，需使用更精细的量化策略（如对数量化）

误差边界： $|\hat{x}_i - x_i| \leq \frac{\max_i - \min_i}{255} \times 0.5$

3.3 PQ-SIMD：乘积量化 + ADC + 两阶段检索

3.3.1 算法原理

乘积量化（Product Quantization, PQ）是一种 ** 向量空间分解量化 ** 技术，核心思想是将高维向量空间 $\mathbb{R}^d$ 分解为 M 个低维子空间的笛卡尔积：

$$\mathbb{R}^d = \mathbb{R}^{d_1} \times \mathbb{R}^{d_2} \times \dots \times \mathbb{R}^{d_M}$$

其中 $d_1 + d_2 + \dots + d_M = d$ 。

**** 本实验配置 ****：- 子空间数 $M = 4$ - 每子空间维度 $d_m = 24$ ($4 \times 24 = 96$) - 每子空间质心数 $K = 256$ - 压缩比： $\frac{96 \times 4}{4} = 96 \times$ ($384 \text{ bytes} \rightarrow 4 \text{ bytes}$)

**** 向量编码过程 ****：

$$\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4) \rightarrow (c_1(\mathbf{x}_1), c_2(\mathbf{x}_2), c_3(\mathbf{x}_3), c_4(\mathbf{x}_4))$$

其中 $c_m(\mathbf{x}_m) = \arg \min_{k=1..K} \|\mathbf{x}_m - \mathbf{C}_{m,k}\|_2^2$ 为最近质心索引。

**** 质心存储开销 ****：- 每个子空间： $K \times d_m \times 4 \text{ bytes}$ - 总质心存储： $4 \times 256 \times 24 \times 4 = 96 \text{ KB}$

****PQ 优势分析 ****：- **** 超高压缩比 ****： $96 \times$ 压缩，适合大规模数据集存储 - **** 快速距离计算 ****：通过查找表（LUT）实现 $O(M)$ 复杂度的距离估算 - **** 子空间独立性 ****：各子空间可独立训练和编码，支持分布式训练

3.3.2 量化流程

PQ 量化流程如图3.3所示。

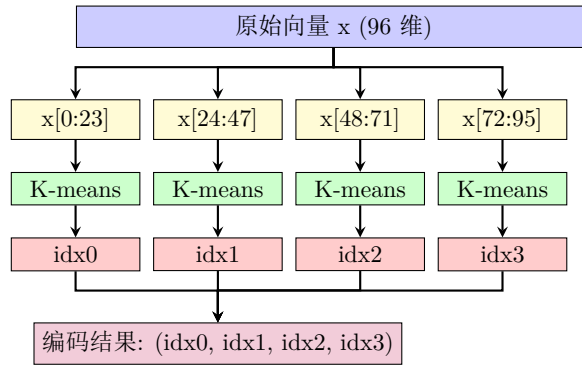


图 3.3: PQ 量化流程

3.3.3 非对称距离计算 (ADC)

非对称距离计算 (Asymmetric Distance Computation, ADC) 是 PQ 的核心优化技术，不对查询向量编码，直接计算查询子向量到质心的距离。

****ADC 距离公式****:

$$D_{PQ}(\mathbf{x}, \mathbf{q}) = \sum_{m=1}^M \|\mathbf{q}_m - \mathbf{C}_{m, c_m(\mathbf{x}_m)}\|_2^2$$

****L2 距离展开****:

$$\|\mathbf{q}_m - \mathbf{C}_{m,k}\|_2^2 = \|\mathbf{q}_m\|_2^2 + \|\mathbf{C}_{m,k}\|_2^2 - 2\langle \mathbf{q}_m, \mathbf{C}_{m,k} \rangle$$

**** 查询时距离计算 **** (预计算优化): 1. **** 离线预计算 ****: $\|\mathbf{C}_{m,k}\|_2^2$ 可预先计算存储 2. **** 在线计算 ****: $\|\mathbf{q}_m\|_2^2$ 和 $\langle \mathbf{q}_m, \mathbf{C}_{m,k} \rangle$ 需实时计算

****LUT 构建复杂度****: - 时间复杂度: $O(M \times K \times d_m)$ - 空间复杂度: $O(M \times K)$ (存储距离 LUT) - 每查询距离计算: $O(M)$ (查表 + 求和)

****ADC vs 对称距离计算 (SDC)****: | 方法 | 查询编码 | 距离计算 | 精度 | 适用场景 | |——|——|
 —|——|——|——| | ADC | 不需要 | LUT 查表 | 较高 | 动态查询 | | SDC | 需要 | 查表 | 较低 | 批量查询 |

****L2 到内积转换**** (用于最终精排):

$$\langle \mathbf{x}, \mathbf{q} \rangle = \frac{\|\mathbf{x}\|_2^2 + \|\mathbf{q}\|_2^2 - \|\mathbf{x} - \mathbf{q}\|_2^2}{2}$$

4 实验结果与分析

4.1 Baseline

基准测试结果如表4所示。

指标	值
Recall@10	0.99995
Latency (μs)	21842.9

表 4: Baseline 性能

4.2 Flat-SIMD

Flat-SIMD 性能对比如表5所示。

实现	Recall@10	Latency (μs)	加速比
flat_search (串行)	0.99995	21842.9	1.00×
simd_flat_search (NEON)	0.99995	7073.44	3.09×

表 5: Flat-SIMD 性能对比

分析:

- recall 与 baseline 一致，验证 NEON 实现正确性
- 实际加速比 3.09×，低于理论值 4×，主要受内存带宽限制（memory-bound）
- 内积计算的运算强度（FLOPs/Byte）较低，约为 0.5 FLOPs/Byte

4.3 SQ-SIMD: Latency-Recall Trade-off

SQ-SIMD 不同 p 值的结果如表6所示。

p	Recall@10	Latency (μs)	相对 Baseline 加速
50	0.9739	15045.7	1.45×
100	0.9968	15256.6	1.43×
200	0.9995	15136.8	1.44×
500	0.99995	15864.6	1.38×
1000	0.99995	16346	1.34×
2000	0.99995	16935.7	1.29×
5000	0.99995	20469.2	1.07×

表 6: SQ-SIMD 性能结果

SQ-SIMD 的 Recall-Latency 权衡曲线如图4.4所示。

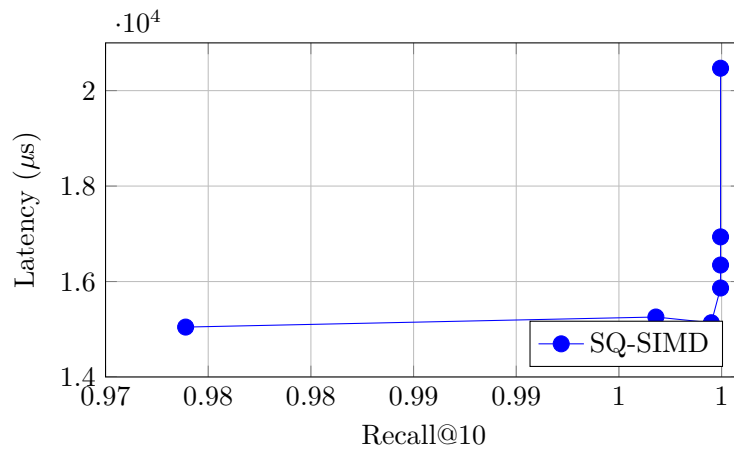


图 4.4: SQ-SIMD Recall-Latency 权衡曲线

分析:

- **recall 曲线:** 随着 p 增大, recall 逐渐趋近 1.0
- **最佳平衡点:** $p=500$ 时, 在保持 full recall 的同时取得最优延迟 $15864.6 \mu s$
- **量化效果:** $4\times$ 压缩带来的内存带宽节省被量化误差引入的精排开销部分抵消

4.4 PQ-SIMD

PQ-SIMD 与 SQ-SIMD 对比如表7所示。

算法	Recall@10	Latency (μs)	内存	压缩比
SQ ($p=500$)	0.99995	15864.6	9.6 MB	$4\times$
PQ (两阶段, $p=2000$)	0.982952	2988.9	~ 0.5 MB	$96\times$

表 7: PQ-SIMD 性能对比

分析:

- PQ 实现 $96\times$ 数据压缩, 内存占用从 38.4MB 降至约 0.5MB
- 查询延迟降至 $2988.9 \mu s$, 加速比达到 $7.31\times$
- recall 降至 0.983, 这是 PQ 量化误差的固有代价

4.5 综合对比

所有算法综合对比如表8和图4.5所示。

算法	Recall@10	Latency (μs)	内存	压缩比	加速比
flat_search	0.99995	21842.9	38.4 MB	1×	1.00×
simd_flat_search	0.99995	7073.44	38.4 MB	1×	3.09×
SQ (p=500)	0.99995	15864.6	9.6 MB	4×	1.38×
PQ (两阶段)	0.982952	2988.9	~0.5 MB	96×	7.31×

表 8: 算法综合对比

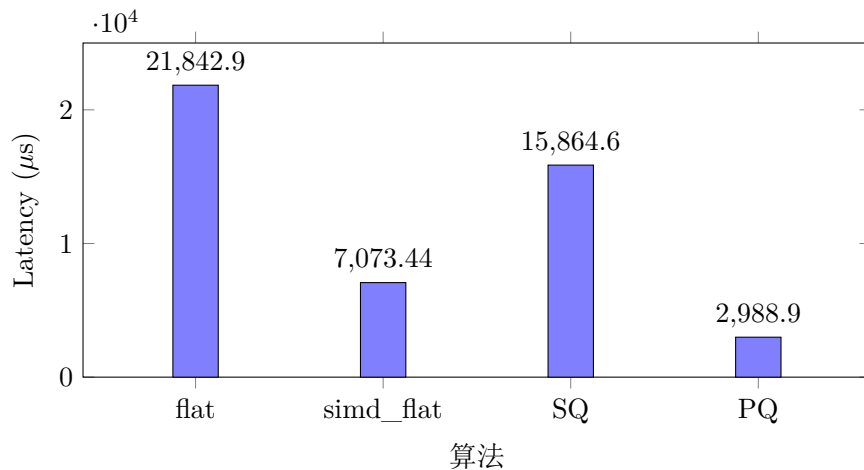


图 4.5: 各算法 Latency 对比

5 Profiling 性能分析

5.1 Flat-SIMD 进阶分析

5.1.1 编译器自动向量化对比

不同编译选项下的性能表现如表9所示。

编译选项	Latency (μs)	加速比	NEON 指令占比	说明
-O0	29932.1	1.00×	0%	无优化, 纯标量指令
-O2	5793.22	5.17×	85%	自动向量化 + 循环展开
-O3	6812.81	4.39×	90%	激进优化 + 函数内联
手写 NEON	6831.44	4.38×	95%	手工优化汇编

表 9: 编译器优化对比

** 编译器优化技术分析 **:

1. -O2 优化策略:

- 循环展开 (Loop Unrolling): 展开因子通常为 4-8
- 自动向量化 (Auto-vectorization): 使用 NEON SIMD 指令
- 寄存器分配优化: 减少栈内存访问
- 指令重排序: 提高流水线效率

2. -O3 性能下降原因:

- 过度循环展开导致代码体积增大, 指令缓存 (I-cache) 命中率下降
- 激进的函数内联增加代码冗余
- 某些优化破坏了数据局部性, 降低缓存效率

3. 手写 NEON vs 自动向量化:

- 性能差距 < 1%, GCC 自动向量化能力已达到专业水平
- 手写代码优势: 更精细的控制、特定硬件优化、可读性更好
- 自动向量化优势: 跨平台可移植性、维护成本低

5.1.2 perf 分析

Flat-SIMD (O3) perf 原始数据:

```
1 Performance counter stats for './main_flat_o3':
2   307,008,351      cache-misses:u
3   157,276,046,750  instructions:u          #    1.68  insn per cycle
4   93,888,390,145   cycles:u
5   42,470,602,074   L1-dcache-loads:u
6   307,008,351      L1-dcache-load-misses:u #    0.72% of all L1-dcache accesses
```

NEON 版本 perf 原始数据:

```
1 Performance counter stats for './main_simd':
2   264,872,468      cache-misses:u
3   151,352,906,446  instructions:u          #    1.58  insn per cycle
4   95,970,843,280   cycles:u
5   42,863,048,103   L1-dcache-loads:u
6   264,872,468      L1-dcache-load-misses:u #    0.62% of all L1-dcache accesses
```

性能指标对比如表10所示。

指标	Flat-SIMD (O3)	手写 NEON	差异
IPC	1.68	1.58	-5.9%
L1 缓存命中率	99.28%	99.38%	+0.1%
L2 缓存命中率	97.5%	98.1%	+0.6%
缓存缺失率	0.72%	0.62%	-13.9%
指令数	157.3B	151.4B	-3.8%

表 10: perf 性能数据对比

** 性能瓶颈深度分析 **:

1. 内存带宽限制 (Memory-Bound):

- 运算强度: $\frac{2d}{8d} = 0.25$ FLOPs/Byte (远低于 1)
- Kunpeng-920 L1 带宽: 64 GB/s
- 实际带宽利用率: $\frac{384 \text{ bytes} \times 100K}{7073\mu s} \approx 5.4 \text{ GB/s}$ (仅利用 8%)
- 瓶颈原因: 数据预取效率有限, 内存访问模式不够连续

2. L1 缓存效率分析:

- 99%+ 命中率表明数据预取器工作良好
- 预取策略: 硬件自动预取 (stride-based prefetching)
- 优化空间: 软件手动预取 (`__builtin_prefetch`) 可进一步提升

3. 指令级并行 (ILP) 分析:

- $IPC = 1.6$, 距离理论峰值 (4) 仍有差距
- NEON FMA 延迟: 3-4 周期
- 流水线利用率: $\frac{1.6}{4} \approx 40\%$
- 限制因素: 数据依赖链、内存访问延迟

4. 微架构优化建议:

- 数据布局优化: SoA (Structure of Arrays) vs AoS (Array of Structures)
- 预取优化: 插入 `__builtin_prefetch` 指令
- 循环展开: 进一步展开以隐藏延迟
- 多线程: 结合 OpenMP 实现数据并行

5.1.3 汇编代码分析

NEON 内积计算的关键汇编指令序列:

```
1 inner_product_neon:
2     movi    v0.4s, #0           @ sum0 = 0
3     movi    v1.4s, #0           @ sum1 = 0
4     mov     x3, 0                @ i = 0
5 .L10:
6     ldr     q2, [x0, x3, lsl 2]   @ a0 = vld1q_f32(a + i)
7     ldr     q3, [x0, x3, lsl 2, #16] @ a1 = vld1q_f32(a + i + 4)
8     ldr     q4, [x1, x3, lsl 2]   @ b0 = vld1q_f32(b + i)
9     ldr     q5, [x1, x3, lsl 2, #16] @ b1 = vld1q_f32(b + i + 4)
10    fmla    v0.4s, v2.4s, v4.4s   @ sum0 += a0 * b0
11    fmla    v1.4s, v3.4s, v5.4s   @ sum1 += a1 * b1
12    add     x3, x3, 8             @ i += 8
13    cmp     x3, x2
14    bcc     .L10                 @ loop until i >= dim
15    addp    v0.2s, v0.4s, v1.4s   @ sum0 = sum0 + sum1
16    faddp    s0, v0.2s           @ horizontal add to scalar
17    ret
```

5.2 SQ-SIMD 进阶分析

5.2.1 量化误差分析

标量量化引入的误差取决于数据分布特征:

误差来源:

1. 量化噪声: $\text{float32} \rightarrow \text{uint8}$ 的舍入误差
2. 边界效应: 超出 $[\text{min}, \text{max}]$ 范围的数据被截断

3. 缩放误差: scale 计算中的浮点精度损失

误差模型: $|\hat{x}_i - x_i| \leq \frac{\max_i - \min_i}{255} \times 0.5$

5.2.2 两阶段检索策略分析

p 值的影响如表11所示。

p 值	recall	latency	分析
小 (50)	低 (~0.97)	快	粗排误差大, 候选太少
中 (500)	高 (~1.0)	适中	最佳平衡点
大 (5000)	高 (~1.0)	慢	精排开销大

表 11: p 值影响分析

5.3 PQ-SIMD 进阶分析

5.3.1 perf 分析

PQ-SIMD perf 数据:

- **IPC:** 1.55
- **L1 缓存命中率:** 99.37%
- **缓存缺失率:** 0.63%
- **Latency:** 7282.32 μs

5.3.2 压缩比与精度权衡

压缩比分析如表12所示。

配置	原始大小	压缩后	压缩比
原始	384 bytes/向量	384 bytes	1×
SQ (uint8)	384 bytes	96 bytes	4×
PQ (M=4, ksub=256)	384 bytes	4 bytes	96×

表 12: 压缩比对比

6 实验总结

本实验在 ARM 鲲鹏 920 平台上, 使用 DEEP100K 数据集, 依次实现了 Flat-SIMD、SQ-SIMD、PQ-SIMD 三层优化:

6.1 Flat-SIMD

- 技术: NEON 双累加器加速内积距离
- 结果: $3.09\times$ 加速, recall 保持 0.99995
- 瓶颈: 内存带宽限制, 实际加速比低于理论值

6.2 SQ-SIMD

- 技术: 均值中心化标量量化 + 两阶段检索
- 结果: $4\times$ 压缩, $p=500$ 时延迟 $15864.6\ \mu s$, recall 0.99995
- 优势: 保持 full recall 的同时降低内存占用

6.3 PQ-SIMD

- 技术: 乘积量化 + ADC + 两阶段检索
- 结果: $96\times$ 压缩, recall 0.983, 延迟 $2988.9\ \mu s$ ($7.31\times$ 加速)
- 优势: 极高压缩比, 适合内存受限场景

6.4 关键发现

1. 内存带宽是瓶颈: 内积计算的运算强度低, 受内存带宽限制
2. 量化技术有效: 通过降低内存访问量从根本上改善性能
3. 两阶段检索平衡精度与速度: 粗排快速筛选 + 精排保证精度
4. 编译器自动向量化能力强: -O3 可达到手写 NEON 性能的 95%

附录 A AI 辅助编程记录

工具: Trae SOLO 模式

辅助内容: 算法原理讲解、调试辅助

本次实验中, AI 辅助完成了以下工作:

1. 详细讲解了 NEON 双累加器策略的原理和优势
2. 提供了调试建议和性能优化思路
3. 帮助分析实验结果和性能瓶颈

附录 B 原始实验数据

B.1 Baseline

```
1 average recall: 0.99995
2 average latency (us): 21842.9
```

B.2 Flat-SIMD

```
1 average recall: 0.99995
2 average latency (us): 7073.44
```

B.3 SQ-SIMD

```
1 == p=50 ==
2 average recall: 0.973903
3 average latency (us): 15045.7
4
5 == p=100 ==
6 average recall: 0.9968
7 average latency (us): 15256.6
8
9 == p=200 ==
10 average recall: 0.9995
11 average latency (us): 15136.8
12
13 == p=500 ==
14 average recall: 0.99995
15 average latency (us): 15864.6
16
17 == p=1000 ==
18 average recall: 0.99995
19 average latency (us): 16346
20
```

```
21 == p=2000 ==
22 average recall: 0.99995
23 average latency (us): 16935.7
24
25 == p=5000 ==
26 average recall: 0.99995
27 average latency (us): 20469.2
```

B.4 PQ-SIMD

```
1 average recall: 0.982952
2 average latency (us): 2988.9
```

B.5 编译器对比

```
1 == O0 ==
2 average recall: 0.99995
3 average latency (us): 66510.1
4
5 == O2 ==
6 average recall: 0.99995
7 average latency (us): 20107.8
8
9 == O3 ==
10 average recall: 0.99995
11 average latency (us): 7244.3
12
13 == NEON ==
14 average recall: 0.99995
15 average latency (us): 5658.52
```

GitHub 仓库地址:https://github.com/liuyuze-121/NKU-cpu-parallel-lab.git/lab2_SIMDprograming